Software Requirements and Design Document

For

Group <1>

Version 1.0

Authors:
Annika Needham
Isabella Maldur
Nikki Zahedi
Jaliah Meade

1. Overview (5 points)

An employee scheduler application that allows managers to automate scheduling processes for their business. Users can log in as an employee or admin/manager. Employees can add their schedule and request to swap shifts. A manager/admin can add/remove employees, generate a master schedule accommodating to everyone's availability. Both can have a calendar view of their schedule.

2. Functional Requirements (10 points)

*List the **functional requirements** in sentences identified by numbers and for each requirement state if it is of high, medium, or low priority. Each functional requirement is something that the system shall do. Include all the details required such that there can be no misinterpretations of the requirements when read. Be very specific about what the system needs to do (not how, just what). You may provide a brief design rationale for any requirement which you feel requires explanation for how and/or why the requirement was derived.*

FR1 (High Priority): The system shall allow users to log in using an email and password.

FR2 (High Priority): The system shall allow users to log out securely.

FR3 (High Priority): The system shall distinguish between employee and administrator roles.

FR4 (High Priority): The system shall allow employees to view their personal schedules.

FR5 (High Priority): The system shall allow employees to submit and edit availability.

FR6 (Medium Priority): The system shall allow employees to request shift swaps.

FR7 (High Priority): The system shall display schedules in a weekly calendar view.

FR8 (Medium Priority): The system shall allow administrators to view all employee schedules.

FR9 (Medium Priority): The system shall allow administrators to add and remove employees.

FR10 (Low Priority): The system shall allow administrators to generate a master schedule automatically.

3. Non-functional Requirements (10 points)

*List the **non-functional requirements** of the system (any requirement referring to a property of the system, such as security, safety, software quality, performance, reliability, etc.) You may provide a brief rationale for any requirement which you feel requires explanation as to how and/or why the requirement was derived.*

NFR1: The system shall securely store user passwords using hashing.

NFR2: The system shall respond to user requests within 2 seconds under normal conditions.

NFR3: The system shall be accessible through modern web browsers.

NFR4: The system shall maintain data integrity in the database.

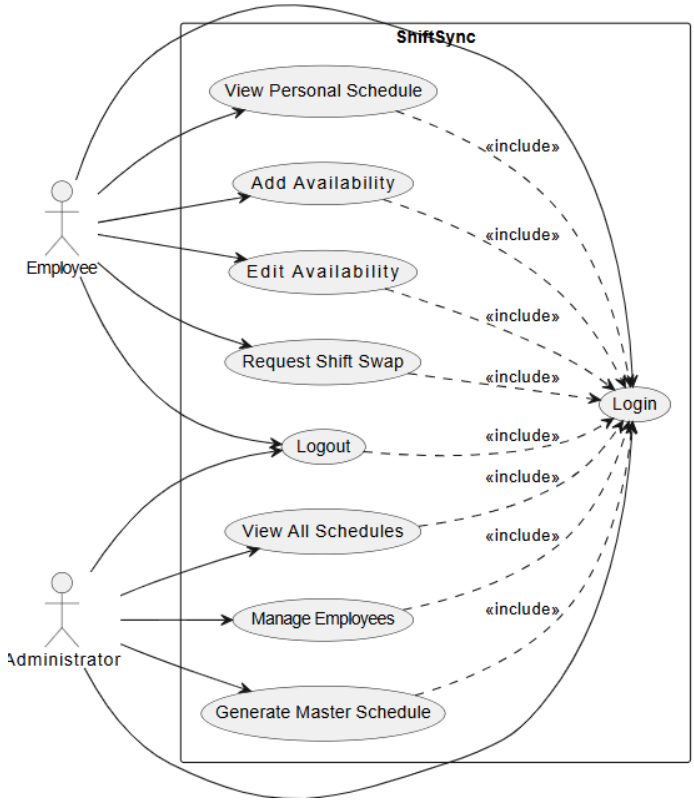
NFR5: The system shall support multiple users simultaneously.

NFR6: The system shall provide error messages for invalid inputs.

Rationale:
These requirements ensure security, reliability, usability, and acceptable performance.

4. Use Case Diagram (10 points)

*This section presents the **use case diagram** and the **textual descriptions** of the use cases for the system under development. The use case diagram should contain all the use cases and relationships between them needed to describe the functionality to be developed. If you discover new use cases between two increments, update the diagram for your future increments.*
***Textual descriptions of use cases**: For the first increment, the textual descriptions for the use cases are not required. However, the textual descriptions for all use cases discovered for your system are required for the second and third iterations.*



5. Class Diagram and/or Sequence Diagrams (15 points)

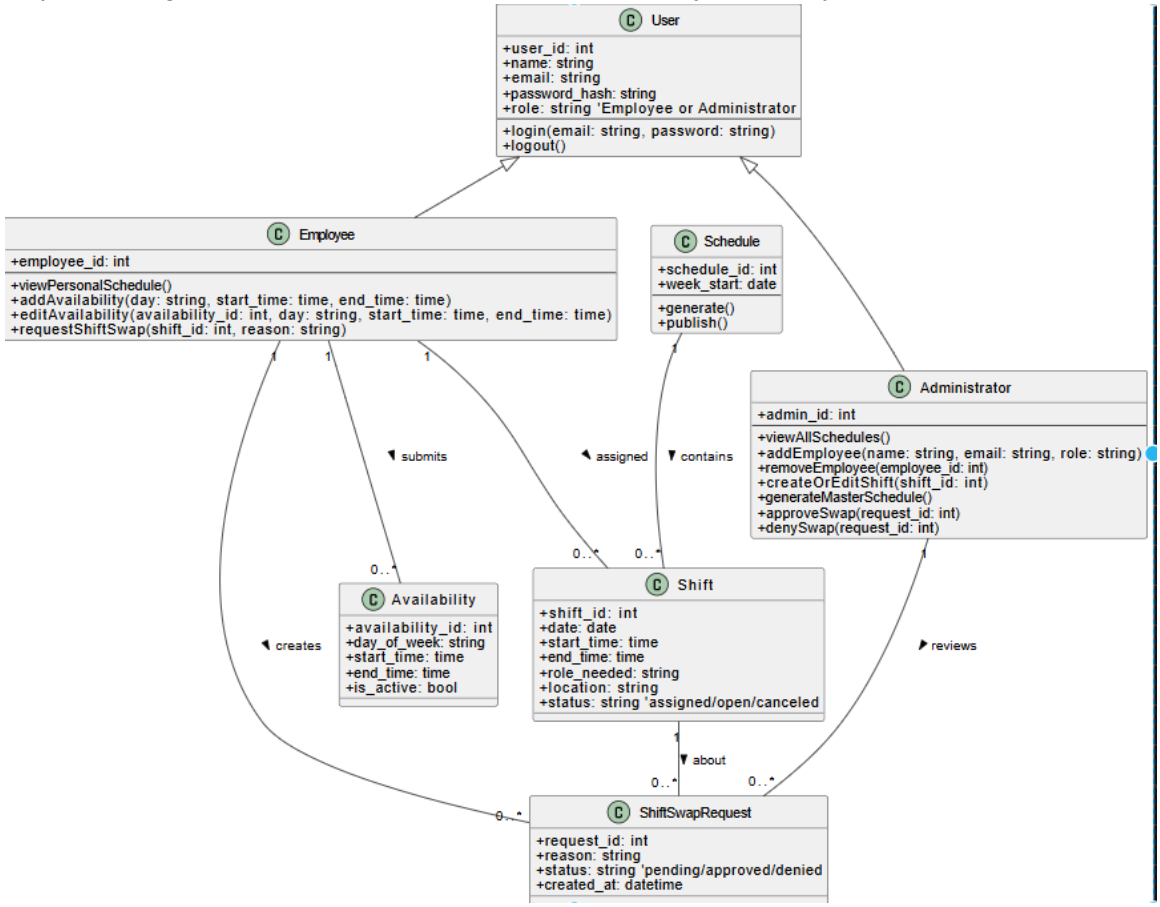
*This section presents a high-level overview of the anticipated system architecture using a **class diagram** and/or **sequence diagrams**.*

*If the main **paradigm** used in your project is **Object Oriented** (i.e., you have classes or something that acts similar to classes in your system), then draw the **Class Diagram of the entire system and Sequence Diagrams for the three (3) most important use cases in your system.***

*If the main **paradigm** in your system is **not Object Oriented** (i.e., you **do not** have classes or anything similar to classes in your system) then only draw **Sequence Diagrams**, **but for all the use cases of your system.** In this case, we will use a modified version of Sequence Diagrams, where instead of objects, the lifelines will represent the functions in the system involved in the action sequence.*

***Class Diagrams** show the **fundamental objects/classes** that must be modeled with the system to satisfy its requirements and **the relationships** between them. Each class rectangle on the diagram **must also include the attributes and the methods of the class** (they can be refined between increments). All the **relationships between classes and their multiplicity** must be shown on the class diagram.*

*A **Sequence Diagram** simply depicts **interaction between objects** (or **functions -** in our case - for non-OOP systems) in a sequential order, i.e. the order in which these interactions take place. Sequence diagrams describe how and in what order the objects in a system function.*



6. Operating Environment (5 points)

Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.

ShiftSync operates in a web-based environment and is accessed in a browser. The application is developed with Python and Flask and can run on Windows, macOS, and Linus. It uses SQLite as database storage and an internet connection is needed.

7. Assumptions and Dependencies (5 points)

List any assumed factors (as opposed to known facts) that could affect the requirements stated in this document. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project.

- **ASSUMPTIONS: Users should have internet connectivity available, Flash & SQLite remain supported**
- **DEPENDENCIES: Flask framework, SQLite database, Python runtime**